ELEC 475: Lab 3 — In-Lab

Pet Nose Localization: SnoutNet, AlexNet and VGG16

Tyler Wilson and Michael Greenspan

Oct. 22, 2026



Table of Contents

Step 1: In-Lab Files 2
Step 2: Generating Predictions 3
Step 3: Error Statistics 4
Step 4: Cumulative Error Distribution 5
Step 5: Best and Worst Predictions 6
Step 6: Ensemble Models 6
Step 7: Completion 7

Step 1: In-Lab Files

Download `lab3_inlab.zip` from the course Drive (https://drive.google.com/drive/folders/1kBLp3StLcN0-GDD17RWMk8WBAXegLsEs) and extract it into your Lab 3 folder, alongside your Pre-Lab code. It contains the code for both the In-Lab and the Post-Lab. The In-Lab uses six of these files:

- `predict.py`: complete. Saves each model's test-set predictions.
- `evaluate.py`: complete. Evaluates the predictions using your functions.
- `utils/profiler.py`: complete. Measures inference time.
- `utils/visualization.py`: complete. Plotting functions used by `evaluate.py`.
- `utils/metrics.py`: to be completed in Steps 3–5.
- `utils/ensemble.py`: to be completed in Step 6.

The remaining files (`train_correction.py`, `correct.py`, `utils/correction_dataset.py` and `models/correction.py`) are for the Post-Lab.

Activate your `.venv`, then confirm that your Pre-Lab code still works:

```
# On macOS / Linux / WSL Bash (Course Standard):
source .venv/bin/activate

# On Windows PowerShell:
.\.venv\Scripts\Activate.ps1

# Confirm your Pre-Lab code still works:
python -m models.snoutnet
```

All three weight files must print `Loaded`. Fix any errors before moving on.

Step 2: Generating Predictions

Each model is run over the 698 test images once, and its predictions are saved to a file.

Run the following line in your terminal:

```
python predict.py --model all
```

For each model, `predict.py` loads the weights from `./checkpoints/`, checks that your dataset's labels match `test_noses.txt`, predicts the nose position in every test image, and measures the inference time per image. The output for each model ends with:

```
[snoutnet] Saved predictions/snoutnet.pt
```

If `predict.py` reports that a label does not match `test_noses.txt`, your `PetNoseDataset` has a bug. The message names the first image with an error.

Each `predictions/<model>.pt` file is a dictionary, loaded with `torch.load(path)`:

Key	Contents
`names`	List of the 698 image file names, in `test_noses.txt` order
`pred`	(698, 2) tensor of predicted (u, v), in original-image pixels
`gt`	(698, 2) tensor of ground-truth (u, v), in original-image pixels
`orig_size`	(698, 2) tensor of original image sizes (W, H)
`params`	Number of model parameters
`ms_per_image`	Average inference time per image, in milliseconds

All coordinates are in original-image pixels.

Step 3: Error Statistics

`evaluate.py` is already complete: it loads the predictions files, calls your functions in `utils/metrics.py` and `utils/ensemble.py`, prints the results and saves the plots. Any function you have not completed yet is skipped, so you should run this script after each step:

```
python evaluate.py
```

Complete the following three functions in `utils/metrics.py`:

Function	Requirements
`euclidean_errors(pred, gt)`	Return an (N,) tensor of the distance between each predicted and true nose position.
`error_stats(errors)`	Return a dictionary with the keys `"min"`, `"mean"`, `"median"`, `"max"` and `"std"`, computed with the PyTorch functions `errors.min()`, `.mean()`, `.median()`, `.max()` and `.std()`.
`baseline_predictions(orig_size, train_file, img_dir)`	Return an (N, 2) tensor of baseline predictions, as described below.

The error for each test image *i* is the Euclidean distance, in original-image pixels:

$$e_i = \sqrt{(\hat{u}_i - u_i)^2 + (\hat{v}_i - v_i)^2}$$

The baseline is the simplest possible model: it uses the average snout position from the training set as its prediction for every test image. Any useful model must beat it. For `baseline_predictions`:

1. For each of the 5,494 training images, compute the normalized snout position (u / W, v / H): the snout position as a fraction of the image width and height. Read the labels from `train_noses.txt`, and get (W, H) with `Image.open(path).size`, which reads only the image header.
2. Average the normalized snout positions over the whole training set. This gives one average normalized snout position, ($\bar{u}$, $\bar{v}$).
3. For each test image, convert the average normalized snout position into a pixel position, ($\bar{u} \times W$, $\bar{v} \times H$), using that image's (W, H) from `orig_size`. This is the baseline's prediction for that image.

Once these functions are complete, `evaluate.py` prints an accuracy table, with errors in original-image pixels:

```
Model                  Min    Mean  Median     Max     Std
Baseline (mean)       xx.x    xx.x    xx.x    xx.x    xx.x
SnoutNet              xx.x    xx.x    xx.x    xx.x    xx.x
SnoutNet-A            xx.x    xx.x    xx.x    xx.x    xx.x
SnoutNet-V            xx.x    xx.x    xx.x    xx.x    xx.x
```

It also prints a cost table:

```
Model                    Params   ms/img
Baseline (mean)               0      0.0
SnoutNet              5,617,794     xx.x
SnoutNet-A           57,012,034     xx.x
SnoutNet-V          134,268,738     xx.x
```

All three models should have a lower mean error than the baseline. If one does not, check that your errors are in original-image pixels, not 227 × 227 pixels.

Step 4: Cumulative Error Distribution

To visually represent each model's performance, you will create a cumulative error distribution (CED) plot, which plots the percentage of predictions (y-axis) that were within a given error (x-axis).

Complete `ced_curve(errors)` in `utils/metrics.py`. It takes an (N,) error tensor and returns two (N,) tensors, x and y:

- x: the errors, sorted in increasing order.
- y: the percentage of predictions with an error of at most x. For the k-th smallest error, this is $100 \times k / N$.

Hint: use `torch.sort`.

`evaluate.py` computes a curve for every row of the accuracy table, plots them together, and saves the plot to `plots/ced.png`.

Step 5: Best and Worst Predictions

Summary statistics hide individual failures. It can be helpful to identify which images each model gets most right and most wrong.

Complete `best_worst_indices(errors)` in `utils/metrics.py`. It takes an (N,) error tensor and returns two tensors of image indices:

- best: the indices of the 4 smallest errors, smallest first.
- worst: the indices of the 4 largest errors, largest first.

Hint: use `torch.argsort`.

For each model, `evaluate.py` draws the best images in the top row and the worst in the bottom row, with the ground truth as a green circle and the prediction as a red cross. It saves one figure per model: `plots/best_worst_snoutnet.png`, `plots/best_worst_alexnet.png` and `plots/best_worst_vgg16.png`.

Step 6: Ensemble Models

An ensemble combines the predictions of several models into one. Complete two functions in `utils/ensemble.py` that each combine the three models' predictions. Both take a list of three (N, 2) prediction tensors and return one (N, 2) tensor:

Function	Requirements
`ensemble_mean(preds)`	Average the three predictions for each image.
`ensemble_median(preds)`	Take the median of the three predictions, separately for u and v.

Run `evaluate.py` again. Both tables gain an `Ensemble (mean)` and an `Ensemble (median)` row, and both ensembles appear on `plots/ced.png`. An ensemble's parameter count and inference time are the sums over its three models, since it must run all three.

Do not adjust the ensembles to improve their test results, for example by weighting each model by its test error. Any choice made by looking at the test results makes those results look better than they really are.

Step 7: Completion

To have your in-lab work marked as complete, you must show a TA the following:

- The accuracy and cost tables printed by `evaluate.py`, with all six rows.
- `plots/ced.png`
- `plots/best_worst_snoutnet.png`, `plots/best_worst_alexnet.png` and `plots/best_worst_vgg16.png`

The marking rubric is as follows:

Item	Mark
Accuracy and cost tables: baseline and model rows	1
`plots/ced.png`	1
`plots/best_worst_*.png`	0.5
Accuracy and cost tables: ensemble rows	0.5
Total	**3**